

Lab project on Information systems discipline

Objectives

As a result of this assignment you will learn basic principles of client-server information systems.

Prerequisites

Courses: Programming, Databases, Web Development.

Instructions

You should create teams of **2-3** classmates to develop the information system. 4 people in the group is too much and not allowed.

The information system to develop should be organized following the principles of three-tier model. Front-end tier as UI for a user to interact with the system typically implemented as web-application or web site. Back-end tier carrying all the domain-specific logic of the system and managing the data, resources, user sessions and accounts. Back-end tier typically runs on a web-server or as a web-service being separated from the front-end host. Data storage tier responsible for data storage and manipulations typically represented as database management system. For example, in the running system, user interacts with front-end using web-browser, which sends requests about different operations about the data to the back-end, which in turn stores the data into the database, queries it to give back to the front-end and demonstrate to the user, and so on.

Requirements for a system

Front-end

Web application should be developed using following technologies for your choice: html5, css/sass/scss, JavaScript (or TypeScript), JSP. If you want to use any front-end framework, you can discuss it with teacher. It is allowable, but not the aim of the course.

Back-end

Spring Boot Java project runs on the web server and communicating with database using **Hibernate ORM**.

Database

PostgreSQL

Steps to accomplish the task

- Stage 1 (10 points) **Deadline March, 30**
 - Create the Use Case diagram for your informational system. The diagram should demonstrate the actors and their actions in system.
 - Create UI sketch showing which pages you are going to organize it with and how are you going to navigate between them.
 - Create the diagram demonstrating database scheme you are going to use for data storage.
 - Choose technologies to use for implementation of the project.
(defaults are: HTML+JS for front-end tier and Spring Framework on Java for back-end tier)

- Send your diagrams along with explanations of the design in the discipline's gitlab to get approve and comments.
- If there are any comments, make modifications according to them.
- Stage 2 (10 points) **Deadline April, 9**
 - Implement REST-service with CRUD (Create, Read, Update, and Delete) operations.
 - Make sure your REST API follows the good practices (see *REST API Guidelines.pdf*)
 - You can use Postman to check how your backend works, while the front-end is not ready.
 - Show your work to the teacher during the practice classes or record a video showing your work, upload to the gitlab and wait for the comments.
- Stage 3 (10 points) **Deadline April, 13**
 - Implement front-end for presenting collection of system's elements, their creation, editing, and deletion and make pagination on the front-end and back-end sides.
 - Make sure your front-end is connected with the back-end. So, when you create a course or ad (depending on the variant), you see this in the database.
 - Make sure that you implemented pagination and files/images view and upload/download according to your variant and functional requirements.
 - Show your work to the teacher during the practice classes or record a video showing your work, upload to the gitlab and wait for the comments.
- Stage 4 (10 points) **Deadline April, 18**
 - Add to existing back-end and front-end users, registration, login forms, the log off button. Changing the password.
 - The password should be encrypted before saving to the database.
 - Logged out user should not be able to paste to the web browser some URL of a page available only for logged in users and get access to it. You need to redirect such request to a login page.
 - Show your work to the teacher during the practice classes or record a video showing your work, upload to the gitlab or send to the dingtalk chat with teacher and wait for the comments.
- Stage 5 (10 points) **Deadline April, 25**
 - Add access control to your system.
 - Show your work to the teacher during the practice classes or record a video showing your work, upload to the gitlab and wait for the comments.

Please complete tasks in time. Each lesson after the **deadline** the max **score** you can get for the task **decreases on one**.

Optional additional tasks (if you want to improve your score):

- Add search or filters (5 points). Note: search and filtering should be implemented **both on front-end and back-end**. Implementing it only on front-end is not a good practice and will not be assessed.
- Add interaction between users of different roles (5 points). You **should discuss** it with your practice instructor before implementing. Examples:
 - Chat
 - For variant 1: possibility for students to apply to the course and for the teacher to give marks for the students

- For variant 2: possibility for regular users to rent the apartment and for landlords to see a rented apartments, who rented them, for how long and how much money this user should pay (accumulate the fee if not payed)

Task variants

Variant 1: Educational courses catalog

The system to be designed should represent an information about the training courses for students. Each teacher giving the particular course can add some information about it into the system. It is required to specify course's name, planned amount of hours for learning activities, brief description text, and course's syllabus file in any format. When teachers submitted a number of courses into the system, any user can browse through the catalog's collection to reach and obtain the information being presented and download syllabus.

The requirements for your project:

1. User should be able to see a catalog of course descriptions.
2. User should see course's name, planned amount of hours for learning activities, brief description text, and course's syllabus file in any format in a menu.
3. Teacher should be able to add any number of courses to the catalog.
4. Teacher should be able to modify his course' information.
5. Teacher should be able to delete his course' information.
6. User should be able to use pagination if there are too many courses in the catalog.
7. User should be able to register in a system.
8. User should be able to log in a system.
9. User should not be able to log in while being logged in.
10. User should be able to log out.
11. User should be able to change password.
12. Administrator should be able to make a student to be a teacher.
13. Administrator should be able to make a *teacher to be a student*.

Variant 2: Rental ads board

The system to design intended to be an ads board for a rental offers. It is targeted for users wanting to rent an apartment or any other residential property. If you are an owner of an unoccupied living apartments or some estate, which you want to rent out, you can use the system to publish an ad on the board, so that it would be available for your potential tenants. A rental offer consists of the address, where the property is located, the description text, and a number of photos being attached.

The requirements for your project:

1. User should be able to see a catalog of ads.
2. User should see address, photo and description of the apartment in a catalog.
3. An apartment owner should be able to create the new ad in a catalog.
4. An apartment owner should be able to modify the ad in a catalog if he created them.
5. An apartment owner should be able to delete the ad if he created them.
6. User should be able to use pagination if there are too many positions in a catalog.
7. User should be able to register in a system.
8. User should be able to log in a system.
9. User should not be able to log in while being logged in.

10. User should be able to log out.
11. User should be able to change password.
12. Administrator should be able to make an apartment owner to be a regular user.
13. Administrator should be able to make a regular user to be an owner.

Hints

- As you are working in a team, you have at least two types of strategy: to divide the tasks and responsibility or to write all the code together following the pair programming styles.
- You do not need to zip your source code to submit it to the gitlab. Try to figure out how to use **GIT** version control system. It will be useful in the future when applying for the job.
- Check that you can build and run your project without compilation or runtime errors before each submission. If your solution cannot run, it cannot be assessed.

REST API guidelines

Basic design guidelines that make a RESTful API:

- ✓ Resources (URIs)
- ✓ HTTP methods
- ✓ HTTP headers
- ✓ Query parameters
- ✓ Status Codes

REST APIs use [Uniform Resource Identifiers](#) (URIs) to address resources. REST API designers should create URIs that convey a REST API's resource model to the potential clients of the API. When resources are named well, an API is intuitive and easy to use. If done poorly, that same API can be challenging to use and understand. *The constraint of a uniform interface is partially addressed by the combination of URIs and HTTP verbs and using them in line with the standards and conventions.*

Resource

In REST, the primary data representation is called **resource**. From the point of API, each entity can be identified as resource. The key abstraction of information in REST is a resource. Any information that can be named can be a resource: a document or image, a temporal service (e.g. "today's weather in Los Angeles"), a collection of other resources, a non-virtual object (e.g., a person), and so on. In other words, any concept that might be the target of an author's hypertext reference must fit within the definition of a resource.

A resource can be a singleton or a collection. For example, "customers" is a collection resource and "customer" is a singleton resource (in a banking domain). We can identify "customers" collection resource using the URI `/customers`. We can identify a single "customer" resource using the URI `/customers/{customerId}`.

A resource may contain sub-collection resources also. For example, sub-collection resource "accounts" of a particular "customer" can be identified using the URN `/customers/{customerId}/accounts` (in a

banking domain). Similarly, a singleton resource “account” inside the sub-collection resource “accounts” can be identified as follows: “/customers/{customerId}/accounts/{accountId}”.

URI case

When it comes to naming resources in a program, there are 3 main types of case conventions: CamelCase, snake_case, and spinal-case.

According to RFC, URLs are “case sensitive” (except for the scheme and the host).

In practice, though, a sensitive case may create dysfunctions with APIs hosted on a Windows system.

It is recommended to use the spinal-case (which is highlighted by RFC3986), this case is used by most big companies.

Example of good REST API

POST /customers for creation with DTO entity in body

GET /customers to get list of customers

GET /customers/{id} to get customer by id

PUT /customers/{id} to edit with DTO entity in body
(not parameters, the whole entity, if it is PUT!)

POST /customers/{id} to edit with html form data

DELETE /customers/{id} for deleting.

HTTP method

As stated earlier, one of the key objectives of the REST approach is using HTTP as an application protocol in order to avoid shaping a homemade API.

Hence, we should systematically use HTTP verbs to describe what actions are performed on the resources and facilitate the developer’s work handling recurrent CRUD operations.

The following methods are usually observed:

GET

The GET method is used to retrieve information from the given server using a given URI. Requests using GET should only retrieve data and should have no other effect on the data.

HEAD

Same as GET, but transfers the status line and header section only.

POST

A POST request is used to send data to the server, for example, customer information, file upload, etc. using HTML forms.

PUT

Replaces all current representations of the target resource with the uploaded content.

PATCH

It is used to make minor updates to resources and it's not required to be idempotent.

DELETE

Removes all current representations of the target resource given by a URI.

OPTIONS

Describes the communication options for the target resource.

Differences between PUT and PATCH

A PUT request will replace the entire content of the resource at the location while a PATCH request, on the other hand, is used to make changes to a part of the resource at a location.

[Query parameters](#)

Often, you will encounter requirements where you will need a collection of resources sorted, filtered, or limited based on some specific resource attribute.

For this requirement, do not create new APIs – instead, enable sorting, filtering, and pagination capabilities in resource collection API and pass the input parameters as query parameters. e.g.

Paging

It is necessary to anticipate the paging of your resources in the early design phase of your API. It is indeed difficult to foresee precisely the progression of the amount of data that will be returned. Therefore, we recommend paginating your resources with default values when they are not provided by the calling client, for example with a range of values [0-25].

Filtering

Filtering consists in restricting the number of queried resources by specifying some attributes and their expected values. It is possible to filter a collection on several attributes at the same time and to allow several values for one filtered attribute.

Sorting

Sorting the result of a query on a collection of resources. A sort parameter should contain the names of the attributes on which the sorting is performed, separated by a comma.

Searching

A search is a sub-resource of a collection. As such, its results will have a different format than the resources and the collection itself. This allows us to add suggestions, corrections, and information related to the search.

Parameters are provided the same way as for a filter, through the query-string, but they are not necessarily exact values, and their syntax permits approximate matching.

Example:

/device-management/managed-devices? page=3& region=USA& filter=region& sort=installation-date&...	Accept: application/json Content-Type: application/json POST http://.../device-management/managed-devices/ searches?page=3 { "terms": {"brand": "XYZ" }, "filter": {"region"}, "sort": {"installation-date"}, ... }
---	---

Status codes

It is very important that as a RESTful API, you make use of the proper HTTP Status Codes. The mostly used status codes are:

200 – OK

Everything is working

201 – CREATED

A new resource has been created

204 – NO CONTENT

The resource was successfully deleted, no response body

304 – NOT MODIFIED

The data returned is cached data (data has not changed)

400 – BAD REQUEST

The request was invalid or cannot be served. The exact error should be explained in the error payload. E.g. „The JSON is not valid “.

401 – UNAUTHORIZED

The request requires user authentication.

403 – FORBIDDEN

The server understood the request but is refusing it or the access is not allowed.

404 – NOT FOUND

There is no resource behind the URI.

500 – INTERNAL SERVER ERROR

API developers should avoid this error. If an error occurs in the global catch block, the stack trace should be logged and not returned as a response.

Best practices

RESTful URI should refer to a resource that is a thing (noun) instead of referring to an action (verb) because nouns have properties that verbs do not have – similarly, resources have attributes. **Use nouns to represent resources.** We should not use URIs to indicate a CRUD function. URIs should only be used to uniquely identify the resources and not any action upon them. We should use HTTP request methods to indicate which CRUD function is performed. **Never use CRUD function names in URIs.**

Some examples of a resource are:

- Users of the system
- User Accounts
- Network Devices etc.

and their resource URIs can be designed as below:

Get all devices	GET /device-management/managed-devices
Create new Device	POST /device-management/managed-devices
Get device for given ID	GET /device-management/managed-devices/{id}
Update device by ID	PUT /device-management/managed-devices/{id}
Delete by ID	DELETE /device-management/managed-devices/{id}

Consistency

Use consistent resource naming conventions and URI formatting for minimum ambiguity and maximum readability and maintainability. You may implement the below design hints to achieve consistency:

Use forward slash (/) to indicate hierarchical relationships

The forward-slash (/) character is used in the path portion of the URI to indicate a hierarchical relationship between resources. e.g.

/device-management/managed-devices/{id}

/device-management/managed-devices/{id}/scripts/{id}

Do not use trailing forward slash (/) in URIs

As the last character within a URI's path, a forward slash (/) adds no semantic value and may confuse. It's better to drop it from the URI.

BAD: /device-management/managed-devices/

GOOD: /device-management/managed-devices

Use hyphens (-) to improve the readability of URIs. Do not use underscores (_)

To make your URIs easy for people to scan and interpret, use the hyphen (-) character to improve the readability of names in long path segments.

GOOD: /inventory-management/**managed-entities**/{id}/**install-script-location**

BAD: /inventory-management/**managedEntities**/{id}/**installScriptLocation**

It's possible to use an underscore in place of a hyphen to be used as a separator – But depending on the application's font, it is possible that the underscore (`_`) character can either get partially obscured or completely hidden in some browsers or screens.

To avoid this confusion, use hyphens (`-`) instead of underscores (`_`).

BAD: `/device_management/managed_devices`

GOOD: `/device-management/managed-devices`

Use lowercase letters in URIs

When convenient, lowercase letters should be consistently preferred in URI paths.

GOOD: `/my-folder/my-doc`

BAD: `/My-Folder/my-doc`

In the above examples, 1 and 2 are the same but 3 is not as it uses **My-Folder** in capital letters.

Do not use file extensions

File extensions look bad and do not add any advantage. Removing them decreases the length of URIs as well. No reason to keep them.

Apart from the above reason, if you want to highlight the media type of API using file extension, then you should rely on the media type, as communicated through the Content-Type header, to determine how to process the body's content.

BAD: `/device-management/managed-devices.xml`

GOOD: `/device-management/managed-devices`

Bad practice

GET `/device-management/get-all-my-managed-devices`

GET `/device-management/managed-devices/getDevice/{id}`

DELETE `/device-management/managed-devices/delete/{id}`